



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 32031704

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 12/10/2025

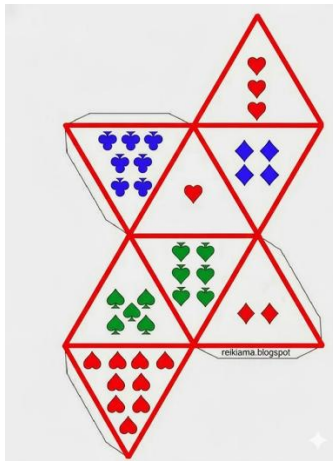
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1: Crear un dado de 8 caras y texturizarlo por medio de código.

Para esta actividad lo primero que hice fue generar una imagen con ayuda de la IA para el dado de 8 caras, tomando como referencia una imagen de internet se genero una con un diseño en sus caras diferente



Posteriormente pasamos a generar los vértices y todo el código para que se generara un octaedro que es un dado de 8 caras.

Creamos la función “CrearDado2”

```
void CrearDado2()
{
    unsigned int cubo_indices2[] = {
        // cara 0
        0, 1, 2,
        // cara 1
        3, 4, 5,
        // cara 2
        6, 7, 8,
        // cara 3
        9, 10, 11,
        // cara 4
        12, 13, 14,
        // cara 5
        15, 16, 17,
        // cara 6
        18, 19, 20,
        // cara 7
        21, 22, 23
    };
};
```

```
// storage meshes
GLfloat cubo_vertices2[] = {

// face 0 (top, front, right) 7 treboles azules
// x      y      z      S      T      NX      NY      NZ
0.0f, 0.5f, 0.0f, 0.26f, 0.50f, 0.577350f, 0.577350f, 0.577350f,
0.0f, 0.0f, 0.5f, 0.51f, 0.75f, 0.577350f, 0.577350f, 0.577350f,
0.5f, 0.0f, 0.0f, 0.01f, 0.75f, 0.577350f, 0.577350f, 0.577350f,

// face 1 (top, right, back) 2-rombos rojos
0.0f, 0.5f, 0.0f, 0.74f, 0.50f, 0.577350f, 0.577350f, -0.577350f,
0.5f, 0.0f, 0.0f, 0.49f, 0.250f, 0.577350f, 0.577350f, -0.577350f,
0.0f, 0.0f, -0.5f, 0.99f, 0.250f, 0.577350f, 0.577350f, -0.577350f,

// face 2 (top, back, left) 3-corazones
0.0f, 0.5f, 0.0f, 0.74f, 1.00f, -0.577350f, 0.577350f, -0.577350f,
0.0f, 0.0f, -0.5f, 0.49f, 0.748f, -0.577350f, 0.577350f, -0.577350f,
-0.5f, 0.0f, 0.0f, 0.99f, 0.748f, -0.577350f, 0.577350f, -0.577350f,

// face 3 (top, left, front) 6 naipes verdes
0.0f, 0.5f, 0.0f, 0.50f, 0.25f, -0.577350f, 0.577350f, 0.577350f,
-0.5f, 0.0f, 0.0f, 0.75f, 0.505f, -0.577350f, 0.577350f, 0.577350f,
0.0f, 0.0f, 0.5f, 0.25f, 0.505f, -0.577350f, 0.577350f, 0.577350f,

// face 4 (bottom, right, front) 4-rombos-azules
0.0f, -0.5f, 0.0f, 0.74f, 0.50f, 0.577350f, -0.577350f, 0.577350f,
0.5f, 0.0f, 0.0f, 0.99f, 0.75f, 0.577350f, -0.577350f, 0.577350f,
0.0f, 0.0f, 0.5f, 0.49f, 0.75f, 0.577350f, -0.577350f, 0.577350f,

// face 5 (bottom, back, right) 5-naipes-verdes-reves
0.0f, -0.5f, 0.0f, 0.26f, 0.51f, 0.577350f, -0.577350f, -0.577350f,
0.0f, 0.0f, -0.5f, 0.01f, 0.255f, 0.577350f, -0.577350f, -0.577350f,
0.5f, 0.0f, 0.0f, 0.51f, 0.255f, 0.577350f, -0.577350f, -0.577350f,

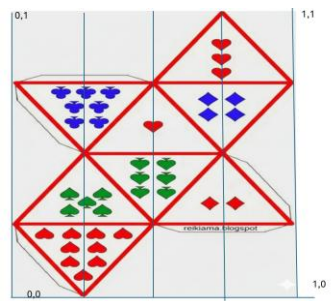
// face 6 (bottom, left, back) 9-corazones rojos
0.0f, -0.5f, 0.0f, 0.255f, 0.01f, -0.577350f, -0.577350f, -0.577350f,
-0.5f, 0.0f, 0.0f, 0.50f, 0.26f, -0.577350f, -0.577350f, -0.577350f,
0.0f, 0.0f, -0.5f, 0.01f, 0.26f, -0.577350f, -0.577350f, -0.577350f,

// face 7 (bottom, front, left)
0.0f, -0.5f, 0.0f, 0.50f, 0.75f, -0.577350f, -0.577350f, 0.577350f,
0.0f, 0.0f, 0.5f, 0.25f, 0.50f, -0.577350f, -0.577350f, 0.577350f,
-0.5f, 0.0f, 0.0f, 0.75f, 0.50f, -0.577350f, -0.577350f, 0.577350f,

};

Mesh* dado2 = new Mesh();
dado2->CreateMesh(cubo_vertices2, cubo_indices2, 192, 24);
meshList.push_back(dado2);
};
```

Tal y como se hizo en el laboratorio ponemos las coordenadas S y T para que el octaedro tenga las texturas en el lugar adecuado.



Con ayuda de la imagen optimizada en gimp y el sistema de referencia adecuado podemos obtener un acercamiento a las coordenadas para texturizar el dado.

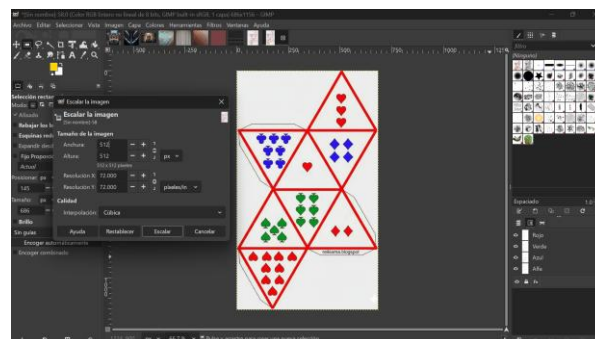


Imagen optimizada con escala potencia de 2 (512 x 512px)

En el código declaramos la textura y hacemos uso de ella

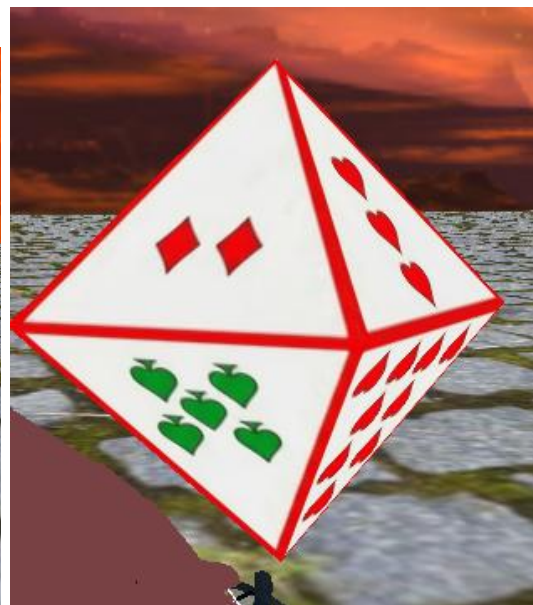
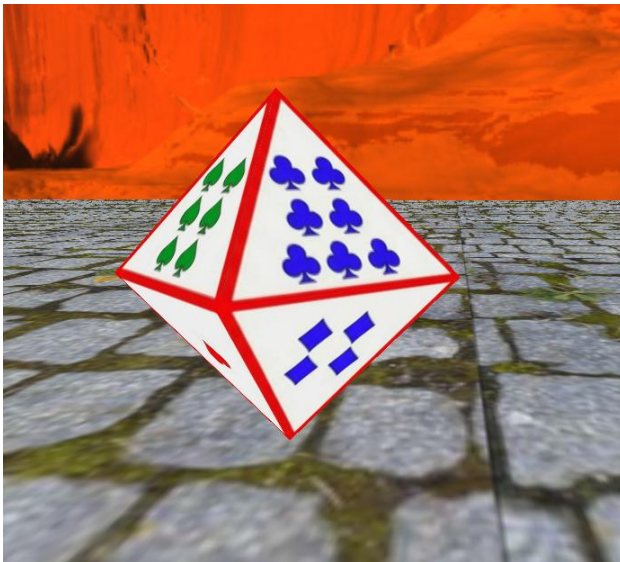
```
Texture logoTexture;  
Texture dado_dosTexture;
```

```
dado_dosTexture = Texture("Textures/dado_ocho_caras_optimizada.png");  
dado_dosTexture.LoadTextureA();
```

Instanciamos el dado

```
//Dado 8 caras  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glEnable(GL_BLEND);  
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);  
dado_dosTexture.UseTexture();  
meshList[5]->RenderMesh();
```

Ejecución

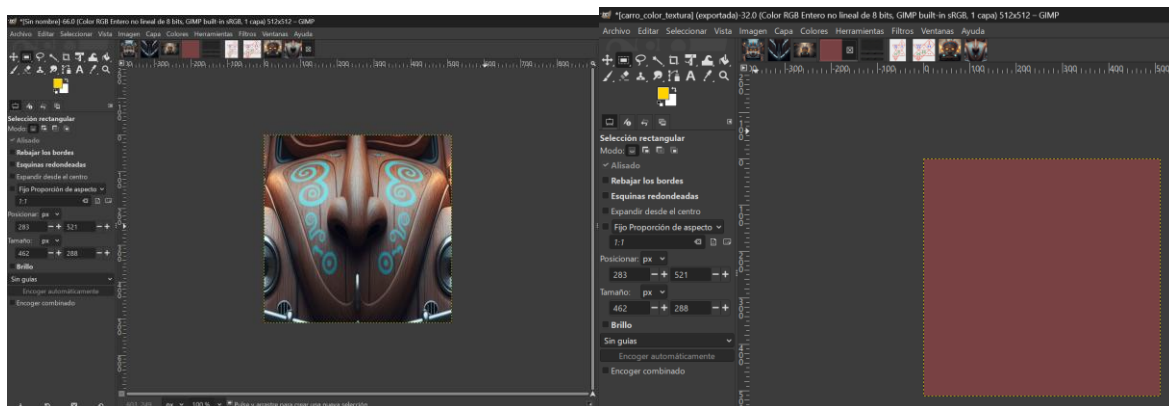


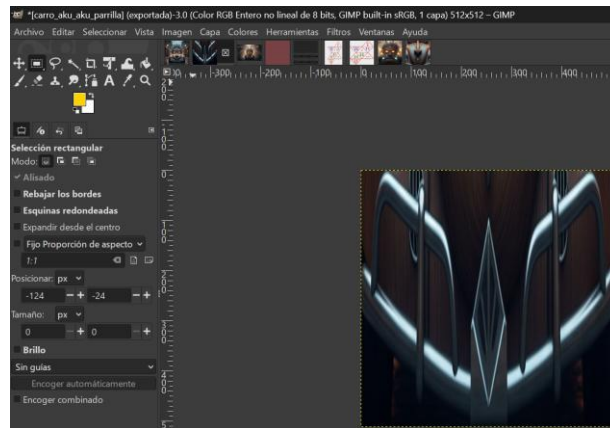
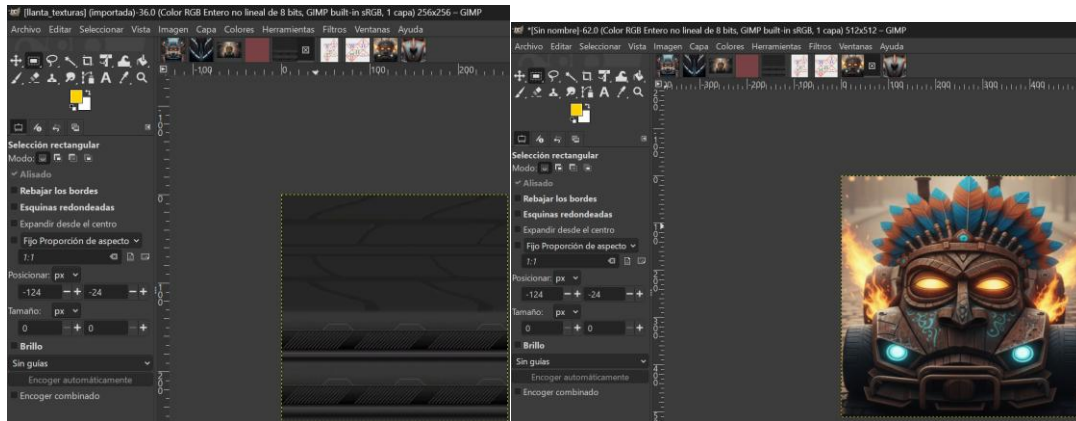
Ejercicio 2: Importar el modelo de su coche con sus 4 llantas acomodadas y tener texturizadas las 4 llantas (diferenciar caucho y rin)

Ejercicio 3: Texturizar la cara del personaje de la imagen tipo cars en el espejo (ojos) y detalles en cofre y parrilla de su propio modelo de coche

Para realizar las dos últimas actividades, primeramente hice uso de los modelos que se generaron en la practica 5 ya que este ya tenia el carro separado de la llantas, con la imagen que se obtuvo del carro tipo cars se texturizaron diferentes partes, en mi caso fue el vidrio para los ojos, el cofre que representa la nariz, los faros que son otros ojos y la defensa ya que la parrilla de mi carro la tiene en la parte de atrás y la imagen solo muestra la parte de enfrente así que los faros fue algo extra.

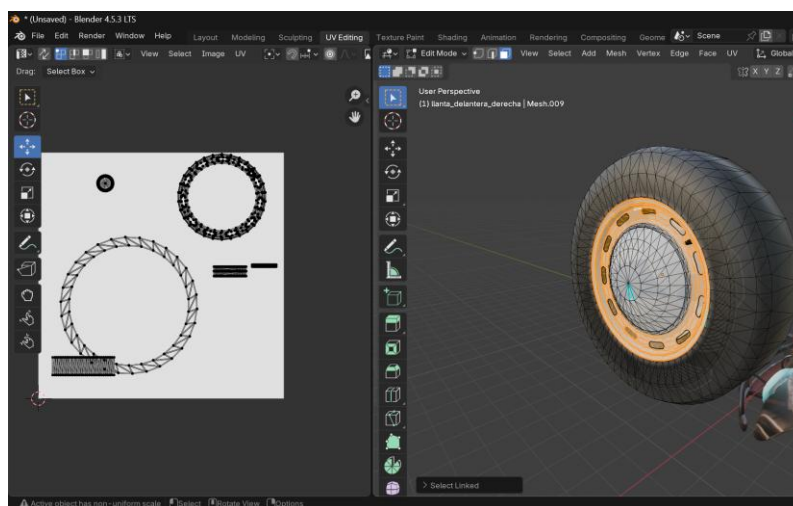
Ahora lo primero que se hizo fue optimizar las imágenes con ayuda de gimp de esta forma se generaron imágenes cuadradas con potencia de 2 (la escala es la más cercana a su escala original).



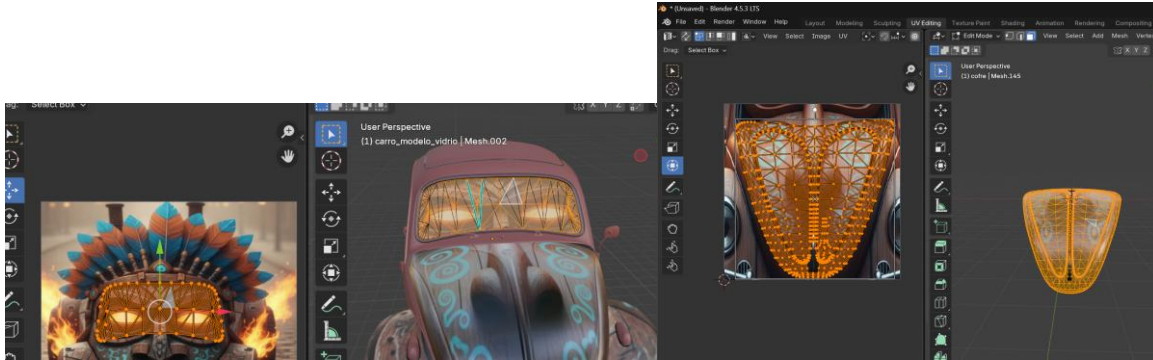


Ya con las imágenes optimizadas en blender pasamos a texturizar cada pieza.

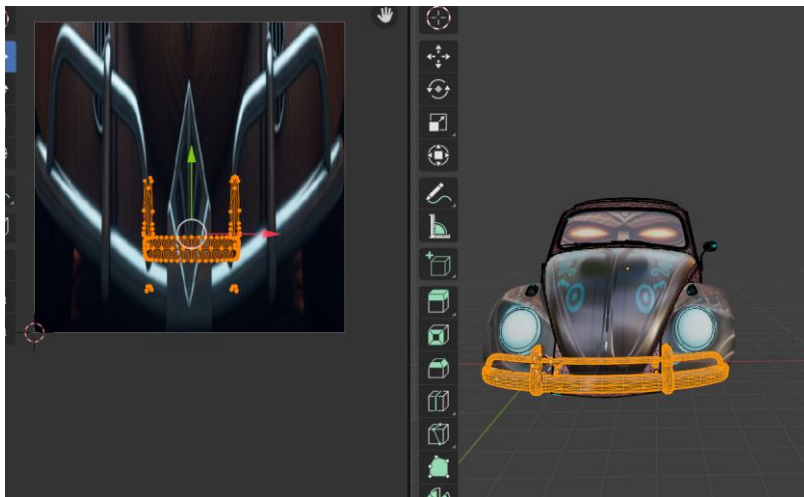
Para llantas utilice las texturas que ya venían junto con mi modelo, lo único que se hizo fue optimizar las imágenes una para el caucho y otra para el rin, esta ultima es solo una imagen en color gris.



Para texturizar la cara.



Defensa.



Cabe señalar que para texturizar el cofre se uso otra imagen que tuviera el mismo modelo de carro para que se viera bien, ya que con el diseño original se deformaba un poco y se veía raro, de ahí en fuera las demás texturas provienen de la imagen original.

Para que el carro tuviera color se uso el color de la madera que tienen las imágenes, con ayuda de blender obtuve el código y en Paint cree una imagen png con el ese color de fondo y así se texturizo todo lo que faltaba del carro.

Una vez que se termino de texturizar, importamos los modelos en formato obj con la configuración que se nos dio y las incluimos en los modelos de las practica para poder hacer uso de ellos en el código.

Creamos los modelos que se van a utilizar para el carro

```
Model Llanta_D;  
Model Llanta_I;  
Model Dado_M;
```

Importamos los modelos que se generaron por medio de blender

```
Carro_M = Model();  
Carro_M.LoadModel("Models/carro_texturas.obj");  
Llanta_D = Model();  
Llanta_D.LoadModel("Models/llanta_derecha.obj");  
Llanta_I = Model();  
Llanta_I.LoadModel("Models/llanta_izquierda.obj");
```

Instanciamos las partes del carro texturizadas

```
//Instancia del coche  
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmuvex() , 1.0f, 8.0f));  
modelaux = model;  
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Carro_M.RenderModel();  
  
//Llanta delantera izquierda  
model = modelaux;  
model = glm::translate(model, glm::vec3(4.3f, -1.5f, 2.0f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));  
color = glm::vec3(0.5f, 0.5f, 0.5f); //Llanta con color gris  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Llanta_I.RenderModel();  
  
//Llanta trasera izquierda  
model = modelaux;  
model = glm::translate(model, glm::vec3(-3.5f, -1.5f, 2.2f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Llanta_I.RenderModel();  
  
//Llanta delantera derecha  
model = modelaux;  
model = glm::translate(model, glm::vec3(4.3f, -1.5f, -2.1f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Llanta_D.RenderModel();  
  
//Llanta trasera derecha  
model = modelaux;  
model = glm::translate(model, glm::vec3(-3.5f, -1.5f, -2.1f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Llanta_D.RenderModel();
```

Ejecución



***Nota:** En la carpeta de modelos y texturas vienen por separado las imágenes de las texturas originales del coche que se deben de incluir en Textures para que no salgan errores en la terminal.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

En si no se si llamarlo problema, ya que no sabía que tenía que agregarle las texturas originales del carro para que no apareciera el error en la terminal.

Otra cosa que ocurrió es que intente usar el código de la practica5 para que el carro se acomodara ya que tal cual son los mismos modelos y aun así no tuve éxito para que se acomodara como debía, por lo que se tuvieron que poner acomodar las coordenadas manualmente.

3.- Conclusión:

Esta práctica me pareció muy tediosa, sobre todo por el uso de blender, ya que considero que la explicación en los videos y el ejercicio fueron muy deficientes para poder realizar la práctica, en esta ocasión los videos no fueron de mucha ayuda, de ahí en fuera considero que aprendí bastante aunque me parece que se puede mejorar bastante sobre todo aplicando la explicación con un ejercicio que se acerque más a lo realizado en la práctica, es decir con un modelo un poco mas complejo, si bien blender es una mejor herramienta para texturizar a comparación de la texturización por medio de código, creo que si es muy tardado sobre todo con modelos que contienen muchas caras como el carro.

Para concluir puedo decir que se logro comprender como texturizar de diferentes formas, así como la importancia de la optimización de las imágenes para un mejor texturizado.